

■ Design Web Crawler

System Design Interview | Search & Aggregation

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional	Crawl web pages starting from seed URLs; Extract links; Download content; Respect robots.txt; Handle redirects; Recrawl for freshness
Non-Functional	Crawl 1B pages in 30 days; Politeness (rate limit per domain); Deduplication; Prioritization; Fault tolerant; Distributed
Scale	$1B/30days = \sim 400 \text{ pages/sec}$; avg page 100KB = 40MB/sec; $1B \times 100KB = 100 \text{ TB/month}$

■ High-Level Architecture

- Frontier (URL Queue) → URL Fetcher → HTML Parser → Link Extractor → Dedup Filter → Frontier
- DNS Resolver cache: batch resolve hostnames; cache TTL 1h; avoid DNS becoming bottleneck
- Politeness scheduler: group URLs by domain; rate limit per domain (e.g., 1 req/sec/domain); use priority queue
- Content store: raw HTML → object storage (S3); extracted links + metadata → Cassandra

■ URL Frontier Design

- **Priority Queue:** Assign priority to URLs based on: PageRank, freshness, domain authority; high-priority URLs fetched sooner
- **Politeness Queue:** Per-domain queues with rate limiting; back-pressure when domain queue is empty; honor Crawl-Delay in robots.txt
- **Two-Queue System:** Front queues (priority-based) → Back queues (domain-based politeness); selector picks from back queue based on domain delay
- **Distributed Frontier:** Kafka as distributed queue; partition by domain hash; each worker owns partition set; enables horizontal scaling

■ Deduplication Strategies

- URL deduplication: Bloom filter (fast, small memory, false positive rate $\sim 0.1\%$); OR consistent hash ring per URL
- Content deduplication: SimHash / MinHash to detect near-duplicate pages; store fingerprint per URL
- URL normalization: lowercase, remove tracking params (?utm_*), resolve relative URLs, canonical URL extraction
- Revisit policy: recrawl based on page change frequency; ETag/Last-Modified headers for conditional GET

■ Politeness & robots.txt

- robots.txt: fetch and cache per domain (TTL 24h); respect Disallow rules and Crawl-Delay directive
- Rate limiting: max 1 req/sec per domain by default; configurable per domain; exponential backoff on errors

- User-Agent: identify crawler with honest User-Agent string; respect X-Robots-Tag meta noindex/nofollow
- Trap detection: infinite URL loops (calendars, session IDs); detect by URL pattern analysis and max depth

■ ■ URL Queue

In-Memory Queue	Distributed Queue (Kafka)
✓ Fast (μ s latency)	✓ Persistent (survives crashes)
✓ Simple implementation	✓ Horizontal scale
✓ Lost on crash	✓ Replay capability
✓ Single machine limit	✓ Higher latency (ms)

■ Handling Scale Challenges

- DNS bottleneck: local DNS cache per crawler node; batch DNS resolution; avoid per-URL DNS lookup
- Hot domains (Wikipedia, GitHub): URL sharding + per-domain rate limiter prevents overwhelming popular sites
- Distributed coordination: Zookeeper assigns URL hash ranges to crawler workers; re-assign on failure
- Checkpoint: periodically snapshot frontier state to S3; resume from checkpoint on system restart